

Automation: Work Queues (Advanced)

Power CAT

Module 05: Cloud flow advanced — work queues (self-exploration modules)

 These are the optional self-exploration modules: Track B, modules B1–B8. Complete the core lab in `05-a-work-queues.md` first, because every module here extends the flows you build there. The appendices referenced throughout live in that same file.

Track B — self-exploration modules

These modules are optional and independent. Each one extends the flows you built in Track A, each states which core module it builds on, and together they complete the full work queue capability set. Attempt them in any order that respects the dependencies noted at the top of each module. None of them introduces a desktop flow — every capability is delivered with cloud flows and the Dataverse connector.

 **Fresher orientation:** How to read Track B: every module is optional and independent, but they mostly edit the **same** processor flow (**Process Order Queue**). Each module starts with a **Builds on** line and, where the steps depend on an earlier module, a **Pick your path** box telling a first-timer exactly where to click.

B1: Enforce a data contract — input schema validation

Malformed work is stopped at the door — validated at enqueue time

Figure: Malformed work is stopped at the door — validated at enqueue time.

Builds on: A1, A3. **Time:** 20–30 min.

 **Fresher orientation:** This module changes no flow. It simply tests the JSON schema you set on the queue in **A1** by trying to add a deliberately bad item by hand — proving the queue rejects malformed work at the door.

You added a JSON schema to the queue in A1. This module proves it works — that the platform validates every item's **Input** at enqueue time and refuses anything that breaks the contract, before malformed work can consume a dequeue, a retry, and an operator's attention.

1. Go to <https://make.powerautomate.com> > **Discover all** > **Monitor** > **Work queues**
Go to <https://make.powerautomate.com> > Discover all > Monitor > Work queues
2. Open **Northwind Order Processing**, and select **See all** in the work queue items section.
Open Northwind Order Processing, and select See all in the work queue items section

Open Northwind Order Processing, and select See all in the work queue items section

3. Select **+ New work queue item** on the toolbar, enter a **Name** of `NW-SCHEMA-TEST`, and in **Input** enter a payload that violates the schema — `OrderValue` sent as text:

```
{ "OrderId": "test", "OrderNumber": "NW-TEST", "OrderValue": "12000" }
```

Leave **Priority** and **Status** unchanged.

Leave Priority and Status unchanged

4. Select **Create**.

✓ **Checkpoint:** The item is **rejected** with a schema validation error and never enters the queue. Input validation happens at enqueue time, so malformed work is stopped at the boundary instead of failing deep inside the processor.

The item is rejected with a schema validation error and never enters the queue

5. Correct the payload — send `12000` as a number, without quotation marks — and select **Create** again. This time the item is accepted.

Correct the payload — send as a number, without quotation marks — and select Create again

Correct the payload — send as a number, without quotation marks — and select Create again

💡 **Tip:** This is exactly why A3 warned you not to quote `OrderValue` in the producer's payload. The contract protects the processor from data it can't process.

This is exactly why A3 warned you not to quote in the producer's payload

This is exactly why A3 warned you not to quote in the producer's payload

B2: Honor the deadline — processing timeout

An item past its deadline is diverted to Processing Timeout, not invoiced late

Figure: An item past its deadline is diverted to Processing Timeout, not invoiced late.

Builds on: A5, A6. **Time:** 25–35 min.

🚀 **Fresher orientation:** This is the first module that edits the processor flow, **Process Order Queue**. Every later B module edits that same flow, so keep it open. Here you add a deadline check so a late item is diverted instead of invoiced. Reminder: the state is **Exception**, and **Processing Timeout** is its reason — see [allowed status transitions](#).

Because A5's FetchXML took over the dequeue order, the orchestrator no longer enforces expiry for you — an item already past its deadline can still be handed to the processor. This module makes the processor check, and diverts a late item to **Processing Timeout** instead of invoicing it late.

1. Open workflow **Process Order Queue** for editing.

Open workflow Process Order Queue for editing

2. Between **Parse Order Item** and **Manager Approval Required**, add a **Condition** and rename it to **Item Still Within Deadline**. In the left value, enter:

```
greater(ticks(outputs('Dequeue_Order_Item')?['body/expirydate']),
ticks(utcNow()))
```

3. Set the operator to **is equal to** and the right value to **true**.

Set the operator to is equal to and the right value

4. Move the approval, invoice, and **Mark Item Processed** actions from A7 into its **True** container.

Move the approval, invoice, and Mark Item Processed actions from A7 into its True container

5. In the **False** container, add a **Microsoft Dataverse Update a row** action and rename it to **Mark Processing Timeout**. Configure it:

- **Table name:** Work Queue Items

- **Row ID:** @{outputs('Dequeue_Order_Item')?['body/workqueueitemid']}

- **Status:** Error

Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.

- **Status Reason:** Processing Timeout

- **Processing Result:** Item passed its expiry date before processing started. Not invoiced.

If certain parameters are not visible they can be seen under the advanced section parameters. Example: Status, Status Reason, Processing Result in the above case.

If certain parameters are not visible they can be seen under the advanced section parameters

Processing timeout is the status for work that failed to complete within its allocated time — a different signal from a business rule violation or a technical fault. Separating it is what lets operations see *lateness* as its own problem with its own fix, such as adding processing capacity rather than correcting data.

6. Select **Save** and **Publish**.

7. **To test:** Create a new item like earlier in B1 to test, once the item's expiry passes before processing starts, it's marked Processing Timeout.

To learn more about SLA management in work queues, see [Create a work queue](#) and [Work queues SLA](#).

To learn more about SLA management in work queues, see [Create a work queue](#) and [Work](#)

To learn more about SLA management in work queues, see [Create a work queue](#) and [Work](#)

✓ **Checkpoint:** Items already past their deadline are diverted to **Processing Timeout** instead of being invoiced late; on-time items flow through to the approval unchanged.

B3: Business exceptions — validation and rejection

A rule (or a person) says no — recorded on the item as a Business Exception

Figure: A rule (or a person) says no — recorded on the item as a Business Exception.

Builds on: A7. **Time:** 30–45 min.



Fresher orientation: Here you wrap the approval with a validation check, so a bad order is recorded as a **Business Exception** instead of failing silently. It edits **Process Order Queue**.



Pick your path: The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you completed B2 (the deadline check)

Your approval, invoice and **Mark Item Processed** now sit inside the **True** container of **Item Still Within Deadline**. Add the new **Order Value Valid** condition there, as the first action in that True container, then move the approval chain into its True container.

If you skipped B2

Your approval, invoice and **Mark Item Processed** sit directly in the **False** container of **Item Dequeued**, right after **Parse Order Item**. Add **Order Value Valid** right after Parse Order Item, then move the approval chain into its True container.

In both paths, then configure the action exactly as the shared steps below describe.

A business exception is work the automation understood perfectly and still cannot complete, because the *data* or the *rule* says no. It is not a bug and retrying will not help — a person has to change something. This module adds two: a payload that fails validation, and an order an approver rejects.

Validate the payload

1. In **Process Order Queue**, add a **Condition** at the location from your path above, and rename it to **Order Value Valid**. Configure it with two rows joined by **And**:
 - **OrderValue** from **Parse Order Item** is greater than 0
 - Expression `empty(body('Parse_Order_Item')['ShipCountry'])` is equal to falseExpression is equal
2. Move the approval, invoice, and **Mark Item Processed** actions into its **True** container.
Move the approval, invoice, and Mark Item Processed actions into its True container
3. In the **False** container, add a **Microsoft Dataverse Update a row** action and rename it to **Mark Business Exception**. Configure it:
 - **Table name:** Work Queue Items
 - **Row ID:** `@{outputs('Dequeue_Order_Item')['body/workqueueitemid']}`
 - **Status:** Error
Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.

- **Status Reason:** Business Exception
- **Processing Result:** Business exception: the order has no value or no ship-to country. Correct the order and requeue.

If certain parameters are not visible they can be seen under the advanced section parameters. Ex: Status, Status Reason, Processing Result in the above case.

If certain parameters are not visible they can be seen under the advanced section parameters

If certain parameters are not visible they can be seen under the advanced section parameters

If certain parameters are not visible they can be seen under the advanced section parameters

If certain parameters are not visible they can be seen under the advanced section parameters

Record an approver rejection

1. In A7 you gave the manager rejection a plain **Terminate**.

In A7 you gave the manager rejection a plain Terminate

2. In the **False** container of **Check Manager Outcome**, Before the terminate action of manager rejected Add a **Microsoft Dataverse Update a row** action and rename it to **Mark Manager Rejection** . Configure it:

- **Table name:** Work Queue Items
- **Row ID:** @{outputs('Dequeue_Order_Item')}['body/workqueueitemid']}
- **Status:** Error
Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.
- **Status Reason:** Business Exception
Processing Result: Business exception: rejected by the manager. No invoice created.

Screenshot for Record an approver rejection

3. Do the same for the executive tier: add **Mark Executive Rejection** before **End Flow - Executive Rejected** . In the **False** container of **Check Executive Outcome**, before the terminate action of executive rejected, add a **Microsoft Dataverse Update a row** action and rename it to **Mark Executive Rejection** . Configure it:

- **Table name:** Work Queue Items
- **Row ID:** @{outputs('Dequeue_Order_Item')}['body/workqueueitemid']}
- **Status:** Error
Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.
- **Status Reason:** Business Exception

Processing Result:

Business exception: rejected by the executive. No invoice created.

Screenshot for Record an approver rejection

Screenshot for Record an approver rejection

 **Important:** A rejection is a **business exception**, not an IT exception — the automation worked exactly as designed and a person decided no. Classifying it correctly matters: IT exceptions are retried automatically, and retrying a rejection would send the same order back to the same approver forever. Marking the item **before** terminating is essential; a terminate alone would leave the item stuck in **Processing** with nothing recorded.

Notice what the processing result is for. It is the message the person who picks this item up will read — so it names the rule that failed and the action required, not a stack trace. Module **B5** puts that message in front of a human.

 **Checkpoint:** Orders that break a business rule or are rejected by an approver are stamped **Business Exception** with an actionable processing result, and no invoice is created for them. Compare this with Module 1, where a bad order simply produced a broken run.

B4: Resilience — IT exceptions, requeue, and delay

A technical blip means try again later — requeue with delay, capped by a budget

Figure: A technical blip means try again later — requeue with delay, capped by a budget.

Builds on: A7 (and the approval/invoice logic). **Time:** 50–70 min. This is the largest optional module — it refactors the processor — so attempt it once the core loop is solid.

 **Fresher orientation:** This is the biggest B module: you wrap the processing in a **Scope** so one catch branch can classify failures as **IT**, **Generic**, or requeue them. Attempt it once the core loop is solid.

 **Pick your path:** The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you built B2 and/or B3

Select from **Parse Order Item** down to **Mark Item Processed** — this range now includes your deadline check (B2) and/or your **Order Value Valid** branch (B3). Group the whole range into the Scope named **Process Order**, so every branch is covered by the catch.

If you built neither

Select from **Parse Order Item** through **Mark Item Processed** — just the approval and invoice — and group that into the Scope named **Process Order**.

Everything in Track A assumes the platform cooperates. It will not always: Dataverse throttles, a connector times out, an approval service hiccups. That class of failure is an **IT exception** — nothing is wrong with the work, only with the moment it was attempted — and the right response is not to escalate to a person but to try again later.

Everything in Track A assumes the platform cooperates

Everything in Track A assumes the platform cooperates

1. In the **False** container of **Item Dequeued**, add an action named **Scope**. Rename the scope to **Process Order**.
2. Move the **Parse Order Item & Item Still Within Deadline** condition inside the scope.
Move the Parse Order Item and Item Still Within Deadline condition inside the scope

For classic designer – Select every action from **Parse Order Item** through **Mark Item Processed**, and group them into a **Scope**. Rename the scope to `Process Order`.

 **Tip:** In the modern designer, add a **Scope** action first and drag the existing actions into it. A scope is a container whose own success or failure summarizes everything inside it — which is what makes a single catch branch possible.

- Below the scope, add a **Compose** action and rename it to `Capture Failure`. In its input, enter:

```
@{result('Process_Order')}
```

Below the scope, add a Compose action and rename it to . In its input, enter

- In the classic designer, open the ... menu on **Capture Failure**, Select **Configure run after**,
In the modern designer, select **Settings** and expand **Process Order**

- Select **has failed**, **is skipped**, and **has timed out**, clear **is successful**.

Modern designer shows the 4 coloured icons as visual cues.

`result()` returns the outcome of every action inside the scope, including each error message — so the catch branch can inspect *what* failed rather than only *that* something failed. That is what makes classification possible instead of guesswork.

Returns the outcome of every action inside the scope, including each error message — so

- Below **Capture Failure**, add a **Condition** and rename it to `Transient Failure`. In the left value, enter the below via fx :

```
or(contains(string(outputs('Capture_Failure')), '429'),  
contains(string(outputs('Capture_Failure')), 'ServiceUnavailable'),  
contains(string(outputs('Capture_Failure')), 'TimedOut'))
```

- Set the operator to **is equal to** and the right value to `true`.

Set the operator to is equal to and the right value

Set the operator to is equal to and the right value

A `429` is Dataverse's service protection limit telling you to slow down; `ServiceUnavailable` and `TimedOut` are the platform saying *not now*. All three are worth another attempt. Anything else is not transient, and repeating it just burns the queue's retry budget.

The transient path: requeue with a delay

- In the **True** container, add a **Condition** and rename it to `Requeue Budget Remaining`. In the left value, enter:

```
int(outputs('Dequeue_Order_Item')?['body/requeuecount'])
```

- Set the operator to **is less than** and the right value to `3` — a deliberate requeue budget this lab enforces in the flow. (The queue's own Item maximum requeue count defaults to 1000); see A1, step 13, for where to view these defaults.

Set the operator to is less than and the right value to — a deliberate requeue

3. In its **True** container, add a **Microsoft Dataverse Update a row** action and rename it to **Requeue Order Item**. Configure it:

Field	Value	Why
Table name	Work Queue Items	The item being requeued.
Row ID	@{outputs('Dequeue_Order_Item')?['body/workqueueitemid']}	The dequeued item.
Status	Queued	Returns the item to the only state it can be dequeued from.
Status Reason	Queued	Matches the status.
Delay until	addMinutes(utcNow(), 15)	The item is invisible to dequeue until this time — the back-off.
Expiry Date	addHours(utcNow(), 4)	Extends the deadline so the retry has a real window.
Requeue Count	add(int(outputs('Dequeue_Order_Item')?['body/requeuecount']), 1)	Consumes one unit of the requeue budget.
Processing Result	Transient failure. Requeued with a 15 minute delay.	Leaves an audit trail on the item.

Delay until is the field that makes a requeue a *back-off* rather than a hot loop. Without it, the item returns to **Queued**, the processor picks it up minutes later, hits the same throttle, and the queue spins. With it, the item is simply not offered for fifteen minutes.

Delay until is the field that makes a requeue a back-off rather than a hot loop

Delay until is the field that makes a requeue a back-off rather than a hot loop

1. In the **False** container of **Requeue Budget Remaining** — the budget is exhausted
Add a **Microsoft Dataverse Update a row** action and rename it to **Mark IT Exception**. Configure it:

- **Table name:** Work Queue Items
- **Row ID:** @{outputs('Dequeue_Order_Item')?['body/workqueueitemid']}
- **Status:** Error
Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.
- **Status Reason:** IT Exception

Processing Result: IT exception: transient failures persisted after 3 requeue attempts. Investigate connectivity or service limits.

Screenshot for The transient path: requeue with a delay

Screenshot for The transient path: requeue with a delay

The unknown path: generic exception

1. In the **False** container of **Transient Failure**

Add a **Microsoft Dataverse Update a row** action and rename it to **Mark Generic Exception**. Configure it:

- **Table name:** `Work Queue Items`
- **Row ID:** `@{outputs('Dequeue_Order_Item')['body/workqueueitemid']}`
- **Status:** `Error`
Error is sometimes referred to as Exception. Refer to [Allowed Status Transitions](#) and [Status codes](#) for more detail.
- **Status Reason:** `Generic Exception`
Processing Result: `Generic exception: @{substring(string(outputs('Capture_Failure')), 0, 1000)}`.
Generic exception is the honest answer for an unexpected error that fits none of the other categories. Recording the raw failure alongside it turns an unknown into something diagnosable.

Generic exception is the honest answer for an unexpected error that fits none of the other

2. Select **Save**, then **Publish**.

 **Tip:** Requeue is your **second** line of defence, not your first. Each Dataverse action has its own retry policy under **Settings** — by default an exponential-interval retry — which absorbs a brief blip inside the same run. The queue's requeue budget is what handles a failure that outlives the run itself.

 **Checkpoint:** A technical failure inside **Process Order** no longer loses the work: transient errors requeue the item with a delay and a consumed budget, exhausted budgets become **IT Exception**, and unclassifiable errors become **Generic Exception** with the raw error attached.

B5: Human in the loop — On hold and remediation

Exceptions get an owner: a reviewer requeues the item or parks it On hold

Figure: Exceptions get an owner: a reviewer requeues the item or parks it On hold.

Builds on: B3 or B4 (there must be exceptions to remediate). **Time:** 40–50 min.

 **Fresher orientation:** Here you add a human step: a reviewer either requeues an exception or parks it **On hold**. This is a new, separate flow — it does not edit **Process Order Queue**.

 **Pick your path:** The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you built B3 or B4

You already produce Exception items. Run **Process Order Queue** once to generate at least one, then continue building the remediation flow below.

If you built neither yet

Create a test exception by hand first: open a queued item on the work queue page and set its **Status** to **Error** (Exception) and its **Status Reason** to **Business Exception** (the allowed [status transitions](#) permit Queued → Exception via the portal). That gives the remediation flow something to find.

Exceptions that a retry cannot fix need an owner. **On hold** is the state that expresses this: an item deliberately paused, not available for processing, waiting for information or for someone to correct it. This module builds the flow that shows exceptions to a person, applies their decision, and returns the item to the queue.

1. Go to [Work queues actions - Power Automate | Microsoft Learn](https://make.powerautomate.com),
Use it to see the status codes and their equivalent numeric values.

Status	Code	Description
Queued	0	Item is queued
Processing	1	Item is being processed
Processed	2	Item was processed
OnHold	3	Item is on hold
Error	4	Item encountered an error

1. Back in <https://make.powerautomate.com>, Navigate to the Order Automation Solution
Back in <https://make.powerautomate.com>, Navigate to the Order Automation Solution
2. Select **New > Automation > Cloud flow > Scheduled cloud flow**, name it **Remediate Order Exceptions**, set it to repeat every **1 Hour**, and select **Create**.
Select **New > Automation > Cloud flow > Scheduled cloud flow**, name it, set
3. Add a **Microsoft Dataverse List rows** action, rename it to **List Exception Items**, set **Table name** to **Work Queue Items**, and in **Filter rows** enter:

```
workqueueid eq <WORK-QUEUE-ID> and statecode eq 4
```

Here **<WORK-QUEUE-ID>** can be found from the Work Queue table with the column name as Work queue, as you copied it in A1 — a value like d1a0a94d-f289-f111-8075-70a8a5af4c98

Here **statecode eq 4** is the Error state or Exception state — the one state shared by every exception reason (Business, IT, Generic, and Processing Timeout), so this filter returns all exception items for review.

Here statecode eq 4 is the Error state or Exception state — the one state shared

4. Add an **Apply to each** loop over **value** from **List Exception Items**, and rename it to **Review Each Exception**.
Add an **Apply to each** loop over value from List Exception Items, and rename it
5. Inside the loop, add **Start and wait for an approval**, rename it to **Exception Review**, and configure it:
 - **Approval type:** **Approve/Reject - First to respond**
 - **Title:** **Work queue exception needs a decision**
 - **Assigned to:** the operations account (yourself, for this lab)

6. In **Details**, put everything the reviewer needs to decide without leaving the email:

```
Item: @{{items('Review_Each_Exception')}['name']}
Result: @{{items('Review_Each_Exception')}['processingresult']}
Requeued: @{{items('Review_Each_Exception')}['requeuecount']} time(s)
Payload: @{{items('Review_Each_Exception')}['input']}
Approve to requeue this item. Reject to park it on hold.
```

In Details, put everything the reviewer needs to decide without leaving the email

7. Below the approval, add a **Condition** named **Reviewer Approved : Outcome is equal to Approve**.

Below the approval, add a Condition named : Outcome is equal

8. In its **True** container, add a **Microsoft Dataverse Update a row** action named **Return Item To Queue**, targeting:

- **Table name:** Work Queue Items
- **Row ID:** @{{items('Review_Each_Exception')}['workqueueitemid']}
- **Status:** Queued
- **Status Reason:** Queued
- **Delay until:** utcNow()
- **Expiry Date:** addHours(utcNow(), 4)
- **Processing Result:** Reviewed and requeued by operations.

Table Work Queue Items Row ID Status Queued, Status Reason Queued, Delay until Expiry Date
Processing

9. In its **False** container, add a **Microsoft Dataverse Update a row** action named **Park Item On Hold**, targeting:

- **Table name:** Work Queue Items
- **Row ID:** @{{items('Review_Each_Exception')}['workqueueitemid']}
- **Status:** On hold
- **Status Reason:** Paused (On hold)
- **Processing Result:** Parked on hold by operations pending further information.

Table Work Queue Items Row ID Status On Hold Status Reason Paused (On Hold) Processing Result

1. Select **Save**, then **Publish**.

The transitions this flow performs are enforced by the platform: an item in **Exception** may go to **Queued** or **On hold**; an item **On hold** may go back to **Queued**; and **Queued** is the only state that can be dequeued. Appendix B lists every allowed transition.

 **Tip:** Because **Allow update item input while in processing** is on for this queue, a remediation flow can also correct the item's **Input** — for example filling in a missing ship-to country — before returning it to the queue, so the retry actually succeeds instead of failing identically.

 **Important:** A work queue item's **Name** and **Input** can't be changed while the item is in the **Processing** state, to protect data integrity mid-run. Remediate items in **Exception** or **On hold**, which is exactly where this flow finds them.

 **Checkpoint:** Exception items are reviewed by a person, and their decision either returns the item to the queue with a fresh deadline or parks it **On hold** — a complete, auditable loop from failure to resolution.

B6: Monitor the SLA

At-risk and violated are different signals — the flow makes both visible before and at breach

Figure: At-risk and violated are different signals — the flow makes both visible before and at breach.

Builds on: A1 (the SLA strategy). **Time:** 25–35 min.

 **Fresher orientation:** Here you turn the queue's SLA into action: an email fires when an item is **At risk** or **Out** of SLA. New, separate flow. More on [work queues SLA](#).

 **Pick your path:** The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you enabled the SLA strategy in A1 (as written)

The **slastatus** column is already being computed on every item — continue and build the monitor flow below.

If you skipped or disabled it

Enable it first: **Monitor > Work queues > Northwind Order Processing > Edit work queue** > turn on the SLA strategy (violated 4h, at-risk 75%), and **Save**. Give it a minute to compute, then continue.

The SLA strategy you configured in A1 continuously computes an **SLA status** on every item: **In**, **At risk**, or **Out**. Those transitions are Dataverse row changes, so a cloud flow can trigger on them and act *before* a breach rather than reporting it afterwards.

1. Back in <https://make.powerautomate.com>, Navigate to the Order Automation Solution
Back in <https://make.powerautomate.com>, Navigate to the Order Automation Solution
2. Select **New > Automation > Cloud flow > Automated cloud flow**, name it **Monitor Order Queue SLA**, choose the **Microsoft Dataverse** trigger **When a row is added, modified or deleted**, and select **Create**.
Select **New > Automation > Cloud flow > Automated cloud flow**, name it, choose
3. Configure the trigger: **Change type Modified Table name Work Queue Items Scope Organization**
Select columns **slastatus**
4. In **Filter rows**, enter:

```
workqueueid eq <WORK-QUEUE-ID> and (slastatus eq 2 or slastatus eq 3)
```

The SLA status values are fixed: 0 **NotSet**, 1 **In**, 2 **At risk**, 3 **Out**. Selecting only `slastatus` in **Select columns** means the flow runs when *that* column changes rather than on every edit to the item.

The SLA status values are fixed: NotSet, In, At risk, Out

5. Add a **Condition** and rename it to `SLA Violated`: the **SLA Status** value is equal to 3 .
6. In the **False** container — at risk but not yet breached — add a **Send an email (V2)** action named `Notify SLA At Risk`, **to**: the operations account (or yourself) **body**: Kindly check **subject**:

```
At risk: @{{triggerOutputs()?['body/name']}} in Northwind Order Processing
```

In the False container — at risk but not yet breached — add a Send

7. In the **True** container, add a **Send an email (V2)** action named `Notify SLA Violated`, **to**: the operations account (or yourself) **body**: Kindly check **subject**:

```
SLA breached: @{{triggerOutputs()?['body/name']}} in Northwind Order Processing
```

In the True container, add a Send an email (V2) action named , **to**: the operations

8. Select **Save**, then **Publish**.

The two signals deserve different responses. **At risk** is a capacity question — the answer is to shorten the processor's recurrence or raise its concurrency (module **B7**). **Out** is an escalation — the deadline is gone and someone must decide whether to still process the item or handle it manually.

 **Tip:** The queue's **Continue to process item even if SLA is violated** setting decides what the orchestrator does with a breached item by default. Either way, this flow is what makes the breach *visible*.

 **Checkpoint:** Items crossing into **At risk** or **Out** of SLA raise an email to operations, so deadlines are managed proactively instead of discovered after the fact.

B7: Scale the processor — concurrency and throughput

Several items processed at once — with parallelism capped to respect Dataverse limits

Figure: Several items processed at once — with parallelism capped to respect Dataverse limits.

Builds on: A5. **Time:** 20–30 min.

 **Fresher orientation:** Here you let the processor take several items per run, with parallelism **capped at five** to respect Dataverse limits.

 **Pick your path:** The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you built the Item Dequeued guard in A5 (as written)

Parallel iterations that find nothing terminate harmlessly through that condition — continue and enable concurrency below.

If you didn't add that guard

Add it first: a **Condition** right after **Dequeue Order Item** that ends the run when no item comes back (see **A5**). Without it, empty parallel iterations error instead of ending cleanly.

One item every five minutes is a teaching cadence, not a working one. Month end needs several items processed at once — but a queue is a shared resource, and turning parallelism up without limit simply moves the bottleneck into Dataverse's service protection limits.

1. Open **Process Order Queue** and change the recurrence to repeat every **1 Minute**.
2. To process several items per run, wrap the dequeue and its processing in an **Apply to each** over a small array and enable concurrency:
 - Next to the trigger Add a **Compose** action named **Worker Slots** with the input `[1,2,3,4,5]`.
Next to the trigger Add a Compose action named with the input
 - Next to compose Add an **Apply to each** loop named **Process In Parallel** over **Outputs** from **Worker Slots**.
Next to compose Add an Apply to each loop named over Outputs from Worker Slots
 - Move **Dequeue Order Item** and everything after it inside the loop.
Move Dequeue Order Item and everything after it inside the loop
 - On the loop, open **Settings**, turn on **Concurrency Control**, and set **Degree of Parallelism** to **5**.
On the loop, open Settings, turn on Concurrency Control, and set Degree of Parallelism

Each iteration performs its own dequeue, so five iterations take five different items — the orchestrator guarantees an item is handed to one consumer only. Iterations that find the queue empty terminate harmlessly through the **Item Dequeued** condition you built in A5.

 **Important:** Keep dequeue concurrency at **moderate** levels — **up to five parallel dequeue operations per work queue** is the recommendation. Work queues are built on Dataverse, so Dataverse [service protection API limits](#) apply to every dequeue and update. Push concurrency past what the platform will absorb and you generate the very **429** responses that module **B4**'s requeue logic then has to clean up.

 **Important:** Work queues aren't suited to high-throughput, sub-second scenarios where hundreds or thousands of items must be processed in seconds. If that is your requirement, use a queuing technology built for it, such as Azure Service Bus queues, and keep work queues for governed, monitored business work.

3. Select **Save**, then **Publish**.

 **Checkpoint:** The processor takes up to five items per run at a one-minute cadence, with parallelism deliberately capped to stay inside platform limits.

 **Note:** If you have done all the labs without skipping there would be below error

The power flow's logic app flow template was invalid. The template actions 'End_Flow_-_Executive_Rejected, Mark_Executive_Rejection' are nested at level '9' which exceeds the maximum nesting limit of '8'.

To address this merge the below conditions together with AND operation. So that it all falls into the nesting limit of 8

To address this merge the below conditions together with AND operation

To address this merge the below conditions together with AND operation

B8: Deep operations, ALM, and the full test matrix

From working to operable — monitored, deployable, and tested on every path

Figure: From working to operable — monitored, deployable, and tested on every path.

Builds on: all modules you have built. **Time:** 50–90 min.

 **Fresher orientation:** This module turns the automation from working into operable: deeper monitoring, deployment hygiene, and a disciplined test pass. See the [Automation center](#) for the cross-queue view.

 **Pick your path:** The exact spot depends on which earlier module you built. Both paths end at the same configuration.

If you completed the full B track

Run all 15 scenarios in the test matrix below — each ends in a different terminal status.

If you built only some modules

Run only the rows whose **Module** column matches what you built, and skip the rest. Rows 1–3 (the Track A core) always apply.

This module turns the automation from *working* into *operable*: deeper monitoring, deployment hygiene, and a disciplined test pass across every path you built.

Deeper monitoring

1. Open a work queue item and read the operational fields the platform maintains alongside your own updates:

Field	What it tells you
Retry Count / Requeue Count	How much of the item's budget has been consumed.
SLA Status / SLA Threshold Time	Where the item stands against its deadline.
Processing Start Time / Processing Duration	How long the item waited and how long it took.

Field	What it tells you
Execution Context	The system-managed list of processing attempts with debugging information.

1. Optional — add an explicit audit entry per attempt. In **Process Order Queue**, add a **Microsoft Dataverse Perform a bound action** action with **Table name** **Work Queue Items**, **Action name** **AddWorkQueueItemProcessingHistoryEntry**, and **Row ID** set to the dequeued item's identifier.
2. Open the **Automation center** from the left navigation and review the **Work queues** page for a cross-queue view of volume, throughput, and SLA adherence.

Deployment hygiene

1. Add the **Remediate Order Exceptions** and **Monitor Order Queue SLA** flows to the **Order Automation** solution alongside the two core flows.
2. Create an **environment variable** to hold the work queue ID, and reference it from each flow in place of the literal GUID. This is what lets the solution deploy to test and production without editing a single flow after import.

The full test matrix

Walk every path you built. Each row ends in a different terminal status; rows for modules you skipped simply won't apply.

#	Test	How to stage it	Expected outcome	Module
1	Happy path, no approval	Order under 1,000 USD set to New .	Priority 3; invoiced without approval; Processed .	A7
2	Two-tier approval	Order above 10,000 USD; approve both.	Priority 1, dequeued first; invoiced; Processed .	A7
3	Priority order	Low- and high-value order within a minute.	High-value item dequeued first despite arriving second.	A3/A5
4	Approver rejection	Order above 1,000 USD; Reject the manager.	Business Exception ; no invoice.	B3
5	Business exception	Remove ShipCountry from a queued item's input.	Business Exception with the rule named.	B3
6	Duplicate protection	Save a still- New order a second time.	Second enqueue rejected — order number already used.	A3
7	Schema validation		Rejected at enqueue time.	B1

#	Test	How to stage it	Expected outcome	Module
		Item whose <code>OrderValue</code> is quoted text.		
8	Processing timeout	Set a queued item's Expiry Date in the past.	Processing Timeout ; no invoice.	B2
9	Requeue with delay	Point Create Invoice at a non-existent table.	Item back to Queued , requeue count +1, delayed 15 min.	B4
10	IT exception ceiling	Leave the forced failure for four runs.	After 3 requeues: IT Exception .	B4
11	Generic exception	Force a non-transient error inside the scope.	Generic Exception with the raw error.	B4
12	Remediation	Run remediation and approve the review.	Item returns to Queued and processes next run.	B5
13	On hold	Run remediation and Reject the review.	Item moves to On hold ; no longer dequeued.	B5
14	SLA at risk / violated	Item with a near expiry; let it lapse.	At risk then breached emails arrive.	B6
15	Empty queue	Run the processor with nothing queued.	Ends at End Flow - Queue Empty, Succeeded .	A5

 **Important:** Remember to undo the deliberate breakages from tests 9, 10, and 11 — restore **Create Invoice** to the **Invoices** table and repair any altered expression — before treating the automation as finished.

Resetting between passes: to clear the queue without deleting it, add a **Perform a bound action** action with **Table name** **Work Queues**, **Action name** **ClearWorkQueue**, and **Row ID** set to your work queue ID, then run it once from a manual flow.

 **Important:** Deleting a work queue permanently deletes **all** related records, including every work queue item and its processing history. Clear the queue when you want a clean slate; delete it only when you are finished with the lab.

 **Checkpoint:** Every path you built behaves as expected, the automation deploys cleanly through a solution and an environment variable, and the work queue page is the single monitored view of the whole process.

Challenge: extend the queue

- Add a second queue, **Northwind Order Exceptions**, and have the processor enqueue a summary item there whenever it records a business exception — one queue feeding another, each monitored and prioritized independently.
- Replace the fixed 15-minute back-off in **Requeue Order Item** with an escalating one — `addMinutes(utcNow(), mul(15, add(int(...requeuecount), 1)))` — so each successive retry waits longer.
- Enqueue from a second source — a Power Apps button, an incoming email, or a scheduled catch-up flow — and confirm the processor is completely indifferent to where the work came from. That indifference is the decoupling you built.

Recommended next step

You have a queue that captures, prioritizes, processes, retries, escalates, and monitors Northwind's order work end to end — entirely in the cloud. The natural next step is to put a conversational front end on it: continue with [Module 3: Workflows](#) and let an agent call a workflow that reports an order's queue status, or hand an exception to a person mid-conversation.